Agentic Efficiency Report

agentic-crm · zip import · scan SCN-2026-01-01-0001

42 **Wasteful + PartialScan**
Weighted efficiency verdict

What we found

Overall efficiency score	42 / 100
Verdict	Wasteful (PartialScan)
Files discovered	94
Files parsed	69
Files skipped	25
Tokens analysed	55,038
Estimated monthly token waste	17,534,970
Estimated monthly credit waste	175.35 credits
Estimated monthly dollar waste	$98.19
Savings range (low / high)	$78.55 - $117.83
Duplicate clusters found	2
Oversized context files	3
Overlapping agent groups	1
Review stages inferred	10
Agent-like files detected	46

Category scores

Category	Score	Weight	Penalty	What this means
Redundancy	0/100	25%	-108	2 groups of near-identical files and 4 repeated instruction blocks were found.
Token bloat	33/100	25%	-67	3 context or memory files are over 8,000 tokens.
Review overhead	64/100	20%	-36	10 review or approval steps were inferred from your instructions.
Agent sprawl	61/100	20%	-39	46 agent-style files were found (20 agent roles, 13 skills).
Architecture inefficiency	83/100	10%	-17	9 service directories against 9 hand written source files.

Top 5 waste drivers, in plain language

1. The same instruction block is pasted into 3 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Redundancy · 6,268,000 tokens/month · 62.68 credits · $35.10

2. Context file is very large: memory.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Token bloat · 2,535,200 tokens/month · 25.35 credits · $14.20

3. 10 separate review or approval steps were found

Your instructions describe many checks before work is accepted. Each extra check means another pass over the same material, which costs time and money.

Review overhead · 2,045,100 tokens/month · 20.45 credits · $11.45

4. 46 agent-style files is more than this repo needs

There are many separate agent, skill and prompt files. Each one adds instructions that have to be loaded and kept in sync. Fewer, clearer files cost less and break less often.

Agent sprawl · 1,632,000 tokens/month · 16.32 credits · $9.14

5. Context file is very large: context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Token bloat · 1,553,200 tokens/month · 15.53 credits · $8.70

Savings assumptions and formulas

• estimated_tokens = ceil(character_count / 4)
• Report credits: 1 credit = 100,000 tokens.
• Vendor billing: $1.00 = 100 vendor credits; 1 vendor credit = 2,500 input tokens or 500 output tokens (output costs 5x input).
• Derived rates: $4.00 per 1M input tokens, $20.00 per 1M output tokens.
• Waste is assumed to be 90% input tokens and 10% output tokens.
• Each agent asset is assumed to be loaded 200 times per month.
• Aggregate savings show a +/-20% range.
• Similarity uses normalised text so whitespace-only and case-only changes are ignored.
• Files under 200 characters are excluded from duplicate detection.
• Scoring runs in two tiers. Tier 1 is the seven specified penalty rules with their hard caps. Tier 2 is severity scaling: for each category we measure the share of the monthly agent context budget that the category wastes, and deduct proportionally up to that category's tier 2 cap, with a category that wastes 50% or more of the budget taking the full deduction. Without tier 2 the capped rules alone could never produce a score below about 72, so the Wasteful and Critical bands would be unreachable. Both tiers are itemised in the score ledger.
• Monthly agent context budget for this repository = 10,933,400 tokens (54,667 agent asset tokens x 200 runs per month).

Setting	Value
Tokens per report credit	100,000
Vendor credits per dollar	100.0
Input tokens per vendor credit	2,500
Output tokens per vendor credit	500
$ per 1M input tokens	$4.00
$ per 1M output tokens	$20.00
Agent runs per month	200
Output token share of waste	10%
Aggregate variance	+/-20%
Rates last refreshed	never
Rates source	Product owner supplied defaults (not yet refreshed)

How the score was calculated

Penalty rule	Hits	Points each	Cap	Applied	Category
Near-duplicate agent/context file pair (similarity >= 0.80)	20	5	32	-32	redundancy
Repeated instruction block >= 150 chars in 3+ files	4	4	20	-16	redundancy
Context or memory file over 8,000 tokens	3	6	24	-18	token_bloat
Agent-like files above 12	34	2	20	-20	agent_sprawl
Review/approval stages above 4	6	5	15	-15	review_overhead
Microservice mismatch (>= 8 service dirs and < 120 source files)	1	12	12	-12	architecture_inefficiency
Monolith mismatch (1 service and >= 10 overlapping agent roles)	0	10	10	-0	architecture_inefficiency

Penalty rule	Hits	Points each	Cap	Applied	Category
Severity scaling - share of the monthly agent context budget wasted by duplication	80	0	60	-60	redundancy
Severity scaling - share of the monthly agent context budget wasted by oversized files	41	0	60	-49	token_bloat
Severity scaling - share of the monthly agent context budget wasted on review loops	19	0	55	-21	review_overhead
Severity scaling - share of the monthly agent context budget wasted on extra agents	16	0	60	-19	agent_sprawl
Severity scaling - share of the monthly agent context budget lost to the architecture	5	0	50	-5	architecture_inefficiency

Findings (14 issues)

Issue	Severity	Category	Title	Impacted files	Tokens/mo	Credits/mo	$/mo
ISS-2026-07-29-0006	critical	redundancy	The same instruction block is pasted into 3 files	context/context.md, context/product-context.md, memory/memory.md	6,268,000	62.68	$35.10
ISS-2026-07-29-0007	high	token_bloat	Context file is very large: memory.md	memory/memory.md	2,535,200	25.35	$14.20
ISS-2026-07-29-0011	high	review_overhead	10 separate review or approval steps were found	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md	2,045,100	20.45	$11.45
ISS-2026-07-29-0010	high	agent_sprawl	46 agent-style files is more than this repo needs	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+21 more)	1,632,000	16.32	$9.14
ISS-2026-07-29-0008	high	token_bloat	Context file is very large: context.md	context/context.md	1,553,200	15.53	$8.70
ISS-2026-07-29-0001	high	redundancy	20 files are near copies of each other	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+16 more)	1,055,200	10.55	$5.91
ISS-2026-07-29-0003	medium	redundancy	The same instruction block is pasted into 43 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+39 more)	764,400	7.64	$4.28
ISS-2026-07-29-0012	high	architecture_inefficiency	Split into many services but there is little code in them	services/svc-1, services/svc-2, services/svc-3, services/svc-4 (+5 more)	546,670	5.47	$3.06
ISS-2026-07-29-0009	medium	token_bloat	Context file is very large: product-context.md	context/product-context.md	401,000	4.01	$2.25
ISS-2026-07-29-0004	medium	redundancy	The same instruction block is pasted into 23 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+19 more)	391,600	3.92	$2.19
ISS-2026-07-29-0005	low	redundancy	The same instruction block is pasted into 20 files	agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md (+16 more)	182,400	1.82	$1.02
ISS-2026-07-29-0013	low	agent_sprawl	13 skill files, and some repeat each other	skills/archive.skill.md, skills/classify.skill.md, skills/escalate.skill.md, skills/extract.skill.md (+9 more)	72,000	0.72	$0.40
ISS-2026-07-29-0014	low	review_overhead	10 orchestration files create extra hand-offs	instruction.md, instructions.md, orchestrator.md, prompts/stage-1.prompt.md (+6 more)	50,000	0.50	$0.28
ISS-2026-07-29-0002	low	redundancy	2 files are near copies of each other	instructions.md, orchestrator.md	38,200	0.38	$0.21

Evidence detail

ISS-2026-07-29-0006 — The same instruction block is pasted into 3 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 62678 characters (~15670 tokens) found in: context/context.md, context/product-context.md, memory/memory.md | Sample: "The customer has an established relationship with the platform and has raised several tickets about billing, onboarding and reporting over the last four quarter"
Formula: monthly_waste = block_tokens x (files_containing_block - 1) x runs_per_month
Recommendation: Move the shared block into one file and reference it from the others. (impact Medium, effort Small)

ISS-2026-07-29-0007 — Context file is very large: memory.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Evidence: memory/memory.md is about 20,676 tokens which is 12,676 tokens over the 8,000 token guideline.
Formula: monthly_waste = (file_tokens - 8000) x runs_per_month
Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0011 — 10 separate review or approval steps were found

Your instructions describe many checks before work is accepted. Each extra check means another pass over the same material, which costs time and money.

Evidence: Stages inferred: Architecture review, Code review, Escalation review, Final approval, Human in the loop, Peer review, Product approval, Security review, Self review, Sign-off. Examples: Architecture review -> agents/billing.agent.md; Code review -> agents/billing.agent.md; Escalation review -> agents/billing.agent.md; Final approval -> agents/billing.agent.md
Formula: monthly_waste = extra_review_stages x total_instruction_tokens x runs_per_month x 0.25
Recommendation: Keep at most four review gates. Combine the overlapping ones into a single checklist. (impact High, effort Medium)

ISS-2026-07-29-0010 — 46 agent-style files is more than this repo needs

There are many separate agent, skill and prompt files. Each one adds instructions that have to be loaded and kept in sync. Fewer, clearer files cost less and break less often.

Evidence: 46 agent-like files detected against a healthy guideline of 12. Median agent file size is 240 tokens.
Formula: monthly_waste = extra_agent_files x median_agent_tokens x runs_per_month
Recommendation: Group agents by job. Merge the ones that overlap and delete the ones nobody calls. (impact High, effort Large)

ISS-2026-07-29-0008 — Context file is very large: context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.

Evidence: context/context.md is about 15,766 tokens which is 7,766 tokens over the 8,000 token guideline.
Formula: monthly_waste = (file_tokens - 8000) x runs_per_month
Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0001 — 20 files are near copies of each other

These files say almost the same thing. Every time your agents run, the same words get sent to the model more than once. Keeping one source of truth removes the repeat cost.

Evidence: Group DUP-001 - highest similarity 100%. Files: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ...
Formula: monthly_waste = (sum of duplicate file tokens - largest file tokens) x runs_per_month
Recommendation: Keep the most complete file, delete or shrink the copies, and link to the kept file instead. (impact High, effort Medium)

ISS-2026-07-29-0003 — The same instruction block is pasted into 43 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.

Evidence: Block of 363 characters (~91 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "Before you begin any task you must read the repository conventions file, confirm the ticket number, check that the branch name follows the release naming policy"
Formula: monthly_waste = block_tokens x (files_containing_block - 1) x runs_per_month
Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0012 — Split into many services but there is little code in them

The repo is arranged as many small services, yet the amount of real code is small. Each service still needs its own setup and its own instructions, so the overhead is larger than the work being done.
Evidence: 9 service directories detected with only 9 hand written source files.
Formula: monthly_waste = 5% x total_agent_asset_tokens x runs_per_month
Recommendation: Fold the thin services back together until each one has a clear, separate job. (impact High, effort Large)

ISS-2026-07-29-0009 — Context file is very large: product-context.md

This memory or context file is bigger than a healthy working budget. Large context files are re-read on every run, so most of the cost is paid again and again.
Evidence: context/product-context.md is about 10,005 tokens which is 2,005 tokens over the 8,000 token guideline.
Formula: monthly_waste = (file_tokens - 8000) x runs_per_month
Recommendation: Split this file into a short always-on summary plus detail files that load only when needed. (impact Medium, effort Medium)

ISS-2026-07-29-0004 — The same instruction block is pasted into 23 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.
Evidence: Block of 353 characters (~89 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "Every change moves through the following gates in order: self review, then peer review, then code review by the owning team, then a security review, then archit"
Formula: monthly_waste = block_tokens x (files_containing_block - 1) x runs_per_month
Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0005 — The same instruction block is pasted into 20 files

One block of instructions has been copied into several agent files. Only one copy is needed. The rest is paid for on every run.
Evidence: Block of 190 characters (~48 tokens) found in: agents/billing.agent.md, agents/escalation.agent.md, agents/forecast.agent.md, agents/handoff.agent.md, agents/intake.agent.md, agents/legacy/intake-backup.agent.md ... | Sample: "When finished, write a summary into the shared memory file and notify the orchestrator so the next stage can start. Include the ticket number, the files touched"
Formula: monthly_waste = block_tokens x (files_containing_block - 1) x runs_per_month
Recommendation: Move the shared block into one file and reference it from the others. (impact High, effort Small)

ISS-2026-07-29-0013 — 13 skill files, and some repeat each other

Skill files are meant to be small and specific. When there are many of them and they repeat, the model reads the same guidance several times.
Evidence: 13 skill files detected; 0 of them sit inside near-duplicate groups.
Formula: monthly_waste = duplicated_skill_tokens x runs_per_month x 0.5
Recommendation: Keep one skill file per capability and delete the near copies. (impact Medium, effort Small)

ISS-2026-07-29-0014 — 10 orchestration files create extra hand-offs

There are many files describing how work moves between agents. Every hand-off adds instructions that must be loaded and kept in step with the others.
Evidence: 10 orchestration or prompt files detected, average size 125 tokens.
Formula: monthly_waste = (orchestration_files - 6) x avg_orchestration_tokens x runs_per_month x 0.5
Recommendation: Describe the flow once in a single orchestrator file and let the agents stay simple. (impact Medium, effort Medium)

ISS-2026-07-29-0002 — 2 files are near copies of each other

These files say almost the same thing. Every time your agents run, the same words get sent to the model more than once. Keeping one source of truth removes the repeat cost.
Evidence: Group DUP-002 - highest similarity 84%. Files: instructions.md, orchestrator.md
Formula: monthly_waste = (sum of duplicate file tokens - largest file tokens) x runs_per_month
Recommendation: Keep the most complete file, delete or shrink the copies, and link to the kept file instead. (impact Medium, effort Small)

Recommended actions, ranked by impact then effort

#	Action	Category	Impact	Effort	Tokens/mo saved	$/mo saved
1	Move the shared block into one file and reference it from the others.	Redundancy	High	Small	764,400	$4.28

#	Action	Category	Impact	Effort	Tokens/mo saved	$/mo saved
2	Keep at most four review gates. Combine the overlapping ones into a single checklist.	Review overhead	High	Medium	2,045,100	$11.45
3	Keep the most complete file, delete or shrink the copies, and link to the kept file instead.	Redundancy	High	Medium	1,055,200	$5.91
4	Group agents by job. Merge the ones that overlap and delete the ones nobody calls.	Agent sprawl	High	Large	1,632,000	$9.14
5	Fold the thin services back together until each one has a clear, separate job.	Architecture inefficiency	High	Large	546,670	$3.06
6	Keep one skill file per capability and delete the near copies.	Agent sprawl	Medium	Small	72,000	$0.40
7	Split this file into a short always-on summary plus detail files that load only when needed.	Token bloat	Medium	Medium	2,535,200	$14.20
8	Describe the flow once in a single orchestrator file and let the agents stay simple.	Review overhead	Medium	Medium	50,000	$0.28

File inventory by category

Group	Files	Parsed	Skipped	Tokens
Agents	20	20	0	5,564
Skills	13	13	0	1,403
Context and memory	3	3	0	46,447
Prompt and orchestration	10	10	0	1,253
Source code	9	9	0	90
Diagrams	1	1	0	13
Docs	2	2	0	207
Other text assets	36	11	25	61

Largest 94 files by estimated tokens

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
memory/memory.md	context_memory	Scanned	8	80.8	20,676	-
context/context.md	context_memory	Scanned	7	61.6	15,766	-
context/product-context.md	context_memory	Scanned	5	39.1	10,005	-
agents/legacy/intake-backup.agent.md	agent	Scanned	23	1.1	288	DUP-001
agents/intake.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/legacy/intake-copy.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/legacy/intake-old.agent.md	agent	Scanned	23	1.1	280	DUP-001
agents/escalation.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/legacy/intake-v2.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/onboarding.agent.md	agent	Scanned	23	1.1	279	DUP-001
agents/forecast.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/proposal.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/reporting.agent.md	agent	Scanned	23	1.1	278	DUP-001
agents/billing.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/handoff.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/notes.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/renewal.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/research.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/support.agent.md	agent	Scanned	23	1.1	277	DUP-001
agents/legal.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/pricing.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/triage.agent.md	agent	Scanned	23	1.1	276	DUP-001
agents/qa.agent.md	agent	Scanned	23	1.1	275	DUP-001
orchestrator.md	orchestration	Scanned	15	0.8	206	DUP-002
instructions.md	orchestration	Scanned	7	0.7	191	DUP-002
prompts/stage-1.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-2.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-3.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-4.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-5.prompt.md	orchestration	Scanned	5	0.4	109	-
prompts/stage-6.prompt.md	orchestration	Scanned	5	0.4	109	-
skills/reconcile.skill.md	skill	Scanned	5	0.4	109	-
skills/summarise.skill.md	skill	Scanned	5	0.4	109	-
skills/translate.skill.md	skill	Scanned	5	0.4	109	-
skills/archive.skill.md	skill	Scanned	5	0.4	108	-
skills/classify.skill.md	skill	Scanned	5	0.4	108	-
skills/escalate.skill.md	skill	Scanned	5	0.4	108	-

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
skills/extract.skill.md	skill	Scanned	5	0.4	108	-
skills/schedule.skill.md	skill	Scanned	5	0.4	108	-
skills/validate.skill.md	skill	Scanned	5	0.4	108	-
skills/format.skill.md	skill	Scanned	5	0.4	107	-
skills/legacy/search-old.skill.md	skill	Scanned	5	0.4	107	-
skills/notify.skill.md	skill	Scanned	5	0.4	107	-
skills/search.skill.md	skill	Scanned	5	0.4	107	-
instruction.md	orchestration	Scanned	5	0.4	105	-
README.md	other	Scanned	4	0.4	105	-
docs/guide.md	other	Scanned	4	0.4	102	-
workflow.md	orchestration	Scanned	5	0.4	97	-
config/settings.yaml	other	Scanned	5	0.1	15	-
docs/architecture.mmd	diagram	Scanned	3	0.0	13	-
package.json	other	Scanned	1	0.0	10	-
services/svc-1/index.js	source	Scanned	1	0.0	10	-
services/svc-2/index.js	source	Scanned	1	0.0	10	-
services/svc-3/index.js	source	Scanned	1	0.0	10	-
services/svc-4/index.js	source	Scanned	1	0.0	10	-
services/svc-5/index.js	source	Scanned	1	0.0	10	-
services/svc-6/index.js	source	Scanned	1	0.0	10	-
services/svc-7/index.js	source	Scanned	1	0.0	10	-
services/svc-8/index.js	source	Scanned	1	0.0	10	-
services/svc-9/index.js	source	Scanned	1	0.0	10	-
services/svc-1/package.json	other	Scanned	1	0.0	4	-
services/svc-2/package.json	other	Scanned	1	0.0	4	-
services/svc-3/package.json	other	Scanned	1	0.0	4	-
services/svc-4/package.json	other	Scanned	1	0.0	4	-
services/svc-5/package.json	other	Scanned	1	0.0	4	-
services/svc-6/package.json	other	Scanned	1	0.0	4	-
services/svc-7/package.json	other	Scanned	1	0.0	4	-
services/svc-8/package.json	other	Scanned	1	0.0	4	-
services/svc-9/package.json	other	Scanned	1	0.0	4	-
assets/blob.json	other	Binary	0	0.0	0	-
data-dump.json	other	SkippedOversized	0	6766.5	0	-
assets/asset-0.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-1.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-10.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-11.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-2.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-3.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-4.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-5.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-6.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-

Path	Category	Status	Lines	Size (KB)	Tokens	Dup group
assets/asset-7.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-8.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/asset-9.png.txt.bak	other	SkippedUnsupported	0	0.0	0	-
assets/logo.svg	other	SkippedUnsupported	0	0.0	0	-
assets/notes.docx	other	SkippedUnsupported	0	0.0	0	-
services/svc-1/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-2/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-3/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-4/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-5/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-6/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-7/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-8/Dockerfile	other	SkippedUnsupported	0	0.0	0	-
services/svc-9/Dockerfile	other	SkippedUnsupported	0	0.0	0	-

Skipped files and warnings

Skipped by reason: Binary = 1, SkippedOversized = 1, SkippedUnsupported = 23

Path	Status	Size (KB)	Reason
assets/blob.json	Binary	0.0	File contains binary data
data-dump.json	SkippedOversized	6766.5	File is 6.6 MB which is above the 5 MB parse limit
assets/asset-0.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-1.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-10.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-11.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-2.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-3.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-4.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-5.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-6.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-7.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-8.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list
assets/asset-9.png.txt.bak	SkippedUnsupported	0.0	Extension '.bak' is not in the supported analysis list

Path	Status	Size (KB)	Reason
assets/logo.svg	SkippedUnsupported	0.0	Extension '.svg' is not in the supported analysis list
assets/notes.docx	SkippedUnsupported	0.0	Extension '.docx' is not in the supported analysis list
services/svc-1/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-2/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-3/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-4/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-5/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-6/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-7/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-8/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list
services/svc-9/Dockerfile	SkippedUnsupported	0.0	Extension 'none' is not in the supported analysis list

Draft replacement files

Each draft is a refined version of a file that already exists in your repository. Filenames carry an -optimised suffix so nothing is overwritten.

memory-optimised.md *(from memory/memory.md, 20,676 -> 1,030 tokens, -95.0%)*

```
# Long Term Memory

## Purpose
Always-on core facts about the customer account. Detailed ticket notes live in separate on-demand files and should be
loaded only when directly relevant to the current task.

## Always-On Core

### Account Summary
- The customer has an established relationship with the platform.
- The relationship spans at least the last four quarters of active use.
- The customer is considered a returning, multi-quarter account rather than a new signup.

### Recurring Ticket Themes
Over the last four quarters, the customer has raised several support tickets. The recurring themes are:
- **Billing** — invoices, charges, plan changes, refunds and payment method issues.
- **Onboarding** — initial setup, user provisioning, workspace configuration and training questions.
- **Reporting** — dashboards, exports, metric definitions and scheduled report delivery.

### Handling History
- Each ticket was resolved by a different agent.
- Resolution notes from every agent were copied into this memory file verbatim, which is why the raw history is large
and highly repetitive.
- No single agent currently owns the account end-to-end; treat handoffs as the norm.

### Implications For Agents
- Assume prior context exists; do not ask the customer to re-explain the basics of their account.
- When a new ticket arrives, first classify it as billing, onboarding, reporting, or other.
- If the ticket falls into one of the three recurring themes, load the matching detail file before responding.
- Preserve continuity: reference prior resolutions when relevant, but do not fabricate details that are not in the
loaded notes.

## Detail Sections (Load On Demand)

The full, un-deduplicated ticket notes have been moved out of always-on memory to reduce context cost. Load only the
file that matches the current ticket topic.

### `memory/details/billing-tickets.md`
- Contains all copied agent notes for billing tickets across the last four quarters.
- Load when the current request involves invoices, charges, plans, refunds, taxes or payment methods.
- Expect duplication; de-duplicate mentally before quoting back to the customer.

### `memory/details/onboarding-tickets.md`
- Contains all copied agent notes for onboarding tickets.
- Load when the request involves initial setup, seat provisioning, SSO, workspace configuration, imports or training.
- Use to check whether an onboarding step was already completed in a prior ticket.

### `memory/details/reporting-tickets.md`
- Contains all copied agent notes for reporting tickets.
- Load when the request involves dashboards, metric definitions, CSV or PDF exports, scheduled reports or data
freshness.
- Cross-reference with billing notes if the reporting question touches usage-based charges.

### `memory/details/agent-handoffs.md`
- Contains per-agent resolution summaries and any handoff context between agents.
- Load when the customer references a previous agent by name, or when continuity across multiple tickets matters.
- Use to reconstruct who did what, in what order, without rereading every raw ticket note.

## Rules For Updating This File
- Keep the always-on core short: account summary, recurring themes, handling history, implications.
- Do not paste raw ticket notes back into this file. Append them to the matching detail file instead.
- If a new recurring theme emerges beyond billing, onboarding and reporting, add it to the themes list here and create
a new detail file under `memory/details/`.
- When closing a ticket, write the resolution note into the correct detail file and, if the account-level status
changed, update the core summary in one short sentence.
- Never delete real constraints, tool names, file paths or acceptance criteria during cleanup; only remove duplicated
prose.

## Acceptance Criteria For This Memory File
- Core section is readable in under one minute.
- Every recurring ticket theme in the core has a matching detail file path listed above.
- No duplicated paragraphs remain in the always-on core.
- Agents can answer a new ticket by reading the core plus at most one detail file in the common case.
```

context-optimised.md *(from context/context.md, 15,766 -> 624 tokens, -96.0%)*

```
# Working Context

## Purpose
Always-on core rules and pointers for agents working in the agentic-crm repo. Detailed background lives in on-demand
files so this document stays small.

## Always-on core rules

Before starting any task, every agent must:

- Read the repository conventions file.
- Confirm the ticket number for the work.
- Check that the branch name follows the release naming policy.
- Make sure the change log entry has been drafted.

Hard rules during work:

- Never commit directly to the `main` branch.
- Always run the unit test suite before handoff.
- Always run the lint task before handoff.
- Hand work over to the next agent in the chain only after both checks pass.

## Handoff checklist

Use this short list at the end of every task:

1. Ticket number recorded.
2. Branch name matches the release naming policy.
3. Change log entry drafted.
4. Unit test suite: passing.
5. Lint task: passing.
6. No direct commits to `main`.
7. Next agent identified.

## Account history (summary only)

The customer is an established platform user. Over the last four quarters they have raised several tickets covering:

- Billing
- Onboarding
- Reporting

Each ticket was resolved by a different agent, and the resolution notes were consolidated into the account history
file. Only load the full history when the current task specifically involves this customer's past tickets, billing
disputes, onboarding steps, or reporting configuration.

## On-demand detail files

Load these only when the current task needs them. Do not preload.

- `context/account-history-full.md`
- Full consolidated notes from all past tickets across billing, onboarding, and reporting.
- Load when: working on a follow-up ticket, auditing prior resolutions, or reconciling agent notes.

- `context/conventions.md` (repository conventions)
- Coding style, naming, review rules.
- Load when: writing or reviewing code, or unsure about a convention.

- `context/release-naming-policy.md`
- Branch and release naming rules.
- Load when: creating a branch or preparing a release.

- `context/changelog-format.md`
- Change log entry format and examples.
- Load when: drafting or editing the change log.

## Notes for agents

- Keep this file lean. If new always-on rules appear, add them here as short bullets.
- If a section grows past a few lines, move the detail into an on-demand file and leave a one-line pointer above.
- When in doubt about a rule, prefer the conventions file over memory.
```